

Generalized-CVO: Fast and Correspondence-Free Local Point Cloud Registration with Second Order Riemannian Optimization^{1*}

Ray Zhang², Marcus Greiff², Thomas Lew², John Subosits^{2 †}

Abstract

We propose a fast and correspondence-free local point cloud registration method that leverages geometric surface structure and reproducing kernel Hilbert space (RKHS) embeddings. The method represents point clouds as continuous functions with point-wise anisotropic kernels that encode local geometry. This formulation improves alignment along surface normals while relaxing alignment along tangential directions. To solve the resulting registration problem, we propose a second-order on-manifold optimization scheme with approximate Riemannian Hessians, achieving a speedup of up to 10x over the first-order solvers used in prior correspondence-free RKHS-based methods. We demonstrate improved frame-to-frame LiDAR and RGB-D tracking accuracy across diverse indoor and outdoor datasets. On a LiDAR tracking registration task in the driving domain, we achieve a reduction of $> 55\%$ in both translational and rotational drift in challenging feature-sparse environments. On object registration benchmarks, we show improved robustness over ICP-based methods and further gains when refining global initialization, particularly under moderate misalignment.

1. Introduction

Point cloud registration is the process of aligning two or more 3D point clouds, i.e., sets of points in space that represent the geometric structure of an object or environment, so that they share a common coordinate system [5, 46]. It has been widely adopted in driving [67], robotic autonomy [49], navigation [30], and scene reconstruction [51], especially when other sensors such as GPS are unavailable or unreliable. In robot localization frameworks such as LiDAR and visual odometry, registration is a key component that consists of estimating robot trajectories and spatially consistent world models using sequential point cloud measurements from LiDAR and RGB-D scans [10, 33]. In odometry

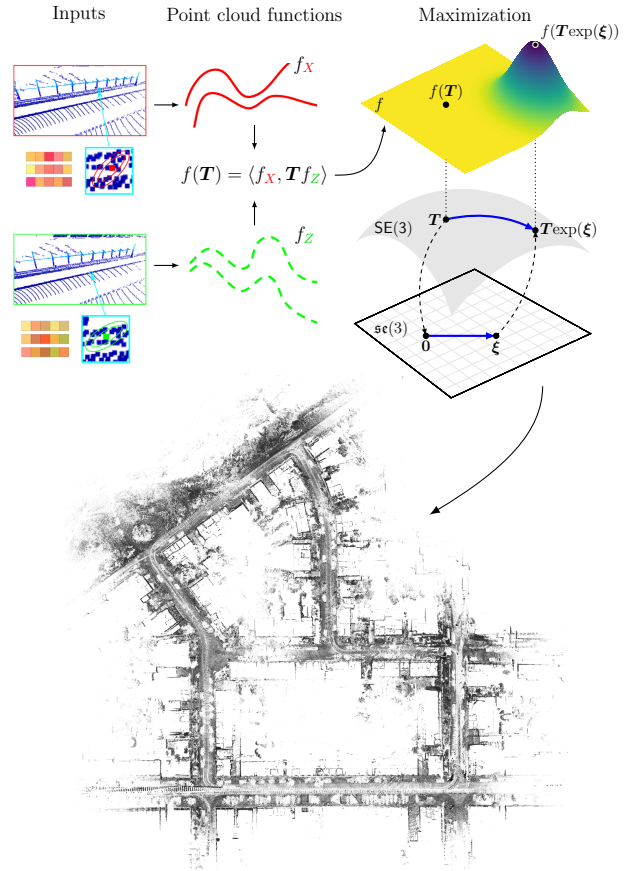


Figure 1. The proposed correspondence-free point cloud registration method in an RKHS using anisotropic kernel embeddings. The registration problem is solved locally via a second-order Riemannian Gauss-Newton optimization. The lower plot shows a stacked point cloud map of KITTI [22] sequence 07, constructed by frame-to-frame tracking without local mapping or loop closure.

settings (often referred to as tracking), registration is commonly initialized with a pose prior that is iteratively refined with respect to a registration objective; this is the primary focus of this work. Such initialization can be obtained by integrating velocity or acceleration measurements [57]. In contrast, global registration methods are designed to handle large initial errors [58, 59] without utilizing pose priors.

Widely adopted in vision-based odometry systems,

¹Project Page: <https://sites.google.com/trl.global/gcvo>

²All authors are with Toyota Research Institute, 4440 El Camino Real, Los Altos, California, USA. {ray.zhang, marcus.greiff, thomas.lew, john.subosits}@trl.global

correspondence-based point cloud registrations may perform well in feature-rich environments [4, 31, 39, 47, 52, 53, 63, 70], but face challenges in feature-sparse settings [69]. Correspondence-based registrations typically use a two-step alternating optimization procedure [12], first identifying nearest neighbors between target and source point clouds (correspondence), followed by finding the relative pose by minimizing aggregated residuals subject to the correspondences [5, 46]. In the correspondence search, the matching process often leverages a spectrum of feature representations, ranging from efficient planar features [46] and bespoke local descriptors [44] to computationally expensive deep learning-based feature embeddings [15, 28, 42]. However, correspondence-based methods rely on the key assumption that point associations can be established reliably, which may not hold in feature-sparse environments such as rural or off-road navigation [69]. Robustness challenges in such environments have motivated the use of robust loss functions [2, 29] and certifiable methods [58].

To tackle challenges in feature-sparse and noisy environments, correspondence-free registration methods such as Continuous Visual Odometry (CVO) [23] have emerged as a promising alternative. In contrast to discrete point sets, correspondence-free methods represent point clouds as continuous functions embedded in a reproducing kernel Hilbert space (RKHS) [16, 23, 65, 66]. This line of work demonstrates robustness to sensor noise and outliers, as it does not rely on known pairwise correspondences [66]. Additionally, it enables the integration of pose-invariant features such as color and pixel labels through the functional representation [64]. However, existing RKHS-based formulations do not exploit local geometric structures often present in point cloud measurements, including surfaces and edges [46, 63]. Furthermore, the cited RKHS registrations rely on 1st-order solver implementations, which are prohibitively slow in applications such as high-performance driving [19, 34].

Contributions: To address these limitations, we propose a correspondence-free objective in an RKHS encoding local geometric information and a second-order on-manifold solver, **Generalized CVO (G-CVO)**. Our RKHS kernel embeddings are implicit and do not rely on additional feature extraction, allowing the registration to fit tightly along surface normals while being less constrained along the surface tangent plane, and resulting in improved robustness and performance in standard registration and tracking metrics [22]. Specifically, we make the following contributions:

- We extend the RKHS loss function in [23] to encode local geometries with anisotropic kernel embeddings.
- We propose a second-order solver using Riemannian Gauss-Newton that is significantly faster than the first-order optimizers in the cited RKHS registration methods.
- Results show that G-CVO achieves superior tracking accuracy and robustness on the urban driving datasets of

KITTI [22], self-collected feature-sparse vehicle-racing datasets, and the indoor ETH RGB-D dataset [45]. Furthermore, G-CVO shows robust ModelNet40 [62] registration under moderate misalignment.

2. Related Works

ICP-based methods are widely used for frame-to-frame registration and consist of an alternating optimization procedure [5, 13, 32, 35, 46]. The first step finds point correspondences, usually with a KD tree [11, 25]. The second step uses an SVD [5] or an on-manifold solver [6, 9] to find an optimal transformation subject to the correspondences. To address the error-prone nature of finding point correspondences in ICP, robust estimation techniques such as M-estimators are often employed to mitigate the impact of large residuals [48]. The robust loss functions can be optimized by graduated non-convexity (GNC) [58] or iteratively reweighted least squares (IRLS) [2]. Unlike the methods above, our approach does not require pairwise correspondences, which may reduce sensitivity to outliers [65].

Another approach to improve robustness is to leverage the local structural information in point clouds. ICP residuals can be reformulated as point-to-feature residuals, such as point-to-plane [13, 46] or point-to-edge [12], which have shown significant improvements in practical LiDAR odometry applications [3, 18, 56, 57, 63]. Some probabilistic approaches model point clouds as Gaussian mixture models (GMM) [7, 20], encoding surfaces and edges by Gaussian distributions. NDT [17, 37, 38] discretizes space and fits Gaussians to each voxel, and then fits a second point cloud to the voxelized collection of Gaussians. Neural networks have been adopted to embed point clouds to capture local invariant and equivariant features in the feature space [28, 36, 41, 42, 61, 72, 73]. Learning-based methods provide rich feature embeddings and coarse correspondences, even under large motions, but a fine-grained registration stage is typically required, either to drive the alignment with these features [14, 58, 60, 68, 71] or to refine it as post-processing [46, 60, 68]. Our method can incorporate such features in the kernel embeddings, or omit feature extraction altogether to favor computational efficiency.

RKHS-based registration addresses the limitations of ICP by removing the need for correspondences [54]. This family of methods includes CVO [23] and Semantic-CVO [64], which represent color and geometric information as functions in RKHS and formulate the registration problem as the maximization of the inner product between two point cloud functions (see Fig. 1). Kernel-Correlation [50] is a special case of CVO that does not use pixel labels. RKHS-BA [66] extends frame-to-frame registration to multi-view bundle adjustment, enabling large-scale mapping. Equiv-Align [65] makes RKHS registration differentiable, enabling end-to-end learning of point-

wise features. Unlike CVO, which uses an isotropic kernel, G-CVO constructs spatially varying anisotropic kernels that adapt to local geometry, improving registration accuracy.

2.1. Notation

Let $X = \{(\mathbf{x}_i, \ell_i^X) \in \mathbb{R}^3 \times \mathbb{R}^l : i \in [N_X]\}$ be a target point cloud or map, and $Z = \{(\mathbf{z}_i, \ell_i^Z) \in \mathbb{R}^3 \times \mathbb{R}^l : i \in [N_Z]\}$ be a new point cloud to be registered, where ℓ_i^j denotes pose-invariant features such as intensity. N_X and N_Z may differ. We denote $\bar{X}$ and $\bar{Z}$ as sets that only contain the 3D coordinates. A pose relating these frames is denoted $\mathbf{T} \in \text{SE}(3)$. Its tangent space is defined as $T_{\mathbf{T}}\text{SE}(3) = \{\mathbf{v} : \mathbf{v} = \gamma'(0), \gamma(0) = \mathbf{T}\}$ where $\gamma : \mathbb{R} \rightarrow \text{SE}(3)$ is a smooth curve passing through $\mathbf{T}$. In particular, its Lie algebra is the tangent space at the identity, i.e. $\mathfrak{se}(3) = T_I\text{SE}(3)$. In the following we use the shorthand $\mathcal{G} = \text{SE}(3)$ with $\mathfrak{g} = \mathfrak{se}(3)$.

The action of a pose in $\mathcal{G}$ on the 3D points is $\mathbf{T}\mathbf{x} = \mathbf{R}\mathbf{x} + \mathbf{t}$, where $\mathbf{R} \in \text{SO}(3)$ and $\mathbf{t} \in \mathbb{R}^3$. We let $(\cdot)^\wedge : \mathbb{R}^6 \mapsto \mathfrak{g}$ denote the *hat* map, mapping a twist vector $\xi = (\omega^\top, \mathbf{v}^\top)^\top$ with rotational and positional components $\omega \in \mathbb{R}^3$ and $\mathbf{v} \in \mathbb{R}^3$, respectively, to the Lie algebra $\mathfrak{g}$. We let the *vee* map $(\cdot)^\vee$ denote the inverse of the hat map. With a slight abuse of notation, we denote the vee map of $\text{SO}(3)$ by $(\cdot)^\vee : \mathfrak{so}(3) \mapsto \mathbb{R}^3$. Then a twist acts on a point $\mathbf{x} \in \mathbb{R}^3$ by $\xi^\wedge \mathbf{x} = \omega^\wedge \mathbf{x} + \mathbf{v}$. The matrix exponential $\exp : \mathfrak{g} \mapsto \mathcal{G}$ maps a twist to a pose, and the logarithm defines its inverse:

$$\exp(\xi^\wedge) = \mathbf{T}, \quad \log(\mathbf{T}) = \xi^\wedge. \quad (1)$$

A vector field $\mathfrak{X}(\mathcal{G})$ assigns one tangent element $U \in T_{\mathbf{T}}\mathcal{G}$ to each $\mathbf{T} \in \mathcal{G}$. To measure distances on the manifold, we consider a Riemannian metric [8], which is a smooth map $\langle \cdot, \cdot \rangle : T_{\mathbf{T}}\mathcal{G} \times T_{\mathbf{T}}\mathcal{G} \mapsto \mathbb{R}$. A Riemannian metric is left-invariant if $\langle U, V \rangle = \langle \mathbf{T}^{-1}U, \mathbf{T}^{-1}V \rangle$, which enables pulling back the metric on $T_{\mathbf{T}}\mathcal{G}$ at $\mathbf{T}$ to the metric on $\mathfrak{g}$ at the identity element. We use a specific left-invariant metric

$$\langle ((\omega_1^\top, \mathbf{v}_1^\top)^\top)^\wedge, ((\omega_2^\top, \mathbf{v}_2^\top)^\top)^\wedge \rangle = \omega_1^\top \omega_2 + \mathbf{v}_1^\top \mathbf{v}_2. \quad (2)$$

Throughout, tangent quantities on $\mathcal{G}$ are left-trivialized by left translation with $\mathbf{T}^{-1}$ onto $\mathfrak{g} \simeq \mathbb{R}^6$ via the vee map $(\cdot)^\vee$.

2.2. Riemannian Gradients and Hessians

To derive Riemannian gradients, we note that for a smooth function $f : \mathcal{G} \rightarrow \mathbb{R}$, and any smooth curve $\gamma : \mathbb{R} \rightarrow \mathcal{G}$ satisfying $\gamma(0) = \mathbf{T} \in \mathcal{G}$ with $\gamma'(0) = U \in T_{\mathbf{T}}\mathcal{G}$, the gradient relates to the directional derivative along U by

$$Df(\mathbf{T})[U] = \langle \text{grad}f(\mathbf{T}), U \rangle = \left. \frac{d}{dt} \right|_{t=0} f(\gamma(t)). \quad (3)$$

We refer to [8] for details, and use the equivalence in (3) when deriving the Riemannian gradients $\text{grad}f$ in Sec. 3.

To derive Hessians, we need to take derivatives with respect to Riemannian gradients. When calculating the directional derivative of $\text{grad}f$, directly subtracting two tangent

vectors is not well-defined when they reside in different tangent spaces. Instead, we use the Levi-Civita connection to project the Hessian onto the tangent space. We denote the connection by $\nabla : \mathfrak{X}(\mathcal{G}) \times \mathfrak{X}(\mathcal{G}) \mapsto \mathfrak{X}(\mathcal{G})$ (see [1, Chapter 5.3] for details), and with it, the Riemannian Hessian is

$$\text{Hess}f(\mathbf{T})[U] = \nabla_U \text{grad}f(\mathbf{T}). \quad (4)$$

Following [1], we express (4) in the context of $\mathcal{G}$ as

$$\nabla_U \text{grad}f = D(\text{grad}f)[U] + \Gamma(U, \text{grad}f). \quad (5)$$

The first term is the directional derivative of the Riemannian gradient $\text{grad}f$ along U . The correction term $\Gamma(U, \text{grad}f)$ in (5) can be derived from the Koszul formula [21, Chapter 21.3] with the left-invariant metric in (2) for $\mathfrak{g}$:

$$\Gamma(U, \text{grad}f) = \frac{1}{2} ([U, \text{grad}f] - (\text{ad}_U^* \text{grad}f)^\vee - (\text{ad}_{\text{grad}f}^* U)^\vee), \quad (6)$$

where $[\cdot, \cdot] : \mathfrak{g} \times \mathfrak{g} \rightarrow \mathfrak{g}$ is the Lie bracket and the adjoint is

$$\text{ad}_{\xi^\wedge} := \begin{bmatrix} \omega^\wedge & \mathbf{0} \\ \mathbf{v}^\wedge & \omega^\wedge \end{bmatrix}, \quad \text{ad}_{\xi^\wedge}^* := \begin{bmatrix} -\omega^\wedge & -\mathbf{v}^\wedge \\ \mathbf{0} & -\omega^\wedge \end{bmatrix}. \quad (7)$$

Let $\mathbf{g}(\mathbf{T}) \triangleq (\text{grad}f(\mathbf{T}))^\vee$ denote the 6D vector representation of $\text{grad}f(\mathbf{T})$. The Hessian in the vector form is

$$(\text{Hess}f(\mathbf{T})[U])^\vee = D\mathbf{g}(\mathbf{T})[U^\vee] + \Gamma^\vee(U, \text{grad}f(\mathbf{T})). \quad (8)$$

2.3. Classical RKHS loss functions

We briefly introduce point cloud registrations in RKHS from CVO [23, 64]. In CVO's formulation, point clouds are represented as continuous functions in an RKHS $\mathcal{H}$ [23] instead of discrete point sets. For a point cloud X , we let

$$f_X(\cdot) = \sum_{(\mathbf{x}, \ell^X) \in X} c(\ell^X) k(\mathbf{x}, \cdot), \quad (9)$$

where $c : \mathbb{R}^l \mapsto \mathbb{R}^+$ is a label function of pose-invariant features, such as LiDAR points' intensities and RGB-D points' colors. Here, $k(\cdot) : \mathbb{R}^3 \times \mathbb{R}^3 \mapsto \mathbb{R}$ is a kernel function. In the following, we denote $\mathbf{T}f_X(\cdot) = f_{\mathbf{T}^{-1}X}(\cdot)$ for $\mathbf{T} \in \mathcal{G}$ as the pose $\mathbf{T}$ acting on the representation f_X of the frame X . The distance in the RKHS between the functions associated with a source frame X and a target frame Z is

$$\begin{aligned} \|f_X - \mathbf{T}f_Z\|_{\mathcal{H}}^2 &= \langle f_X, f_X \rangle + \langle \mathbf{T}f_Z, \mathbf{T}f_Z \rangle - 2\langle f_X, \mathbf{T}f_Z \rangle \\ &= \langle f_X, f_X \rangle + \langle f_Z, f_Z \rangle - 2\langle f_X, \mathbf{T}f_Z \rangle, \end{aligned} \quad (10)$$

To minimize the distance between the functions f_X and f_Z , we maximize the third term containing both point clouds and $\mathbf{T}$. This approach is akin to ICP's objective function, but posed as an optimization problem that no longer requires pair-wise correspondences:

$$\max_{\mathbf{T} \in \mathcal{G}} \langle f_X, \mathbf{T}f_Z \rangle. \quad (11)$$

Using the kernel trick [64], this loss can be simplified as

$$\langle f_X, T f_Z \rangle = \sum_{\substack{(\mathbf{x}_i, \ell^X) \in X \\ (\mathbf{z}_j, \ell^Z) \in Z}} \langle \ell^X, \ell^Z \rangle k(\mathbf{x}_i, T^{-1} \mathbf{z}_j) \triangleq f(\mathbf{T}), \quad (12)$$

with a constant $\langle \ell^X, \ell^Z \rangle$ encoding the intensity similarity of LiDAR points. The isotropic squared exponential kernel is commonly used [16], but we have freedom in choosing the kernel embedding, which we will discuss in Sec. 3. The classical RKHS-based registrations rely on implementations of 1st-order Riemannian gradient ascents.

3. Generalized CVO

To improve convergence compared to prior work discussed in Sec. 2, we propose the G-CVO registration method. It consists of two main components: 1) an anisotropic kernel embedding, inspired by methods that encode local surface structure, and 2) iterative optimizers that use closed-form expressions of the Riemannian gradients and Hessians. Both the 1st- and 2nd-order versions of G-CVO are sketched in Alg. 1.

3.1. Anisotropic covariance

In RKHS-based methods, we have significant freedom in choosing the kernel embedding. Previous formulations proposed the isotropic kernels [54] due to their simplicity and smoothness. Observing that point cloud scans are usually surface scans, we generalize the isotropic kernels in [23] to anisotropic exponential kernels that model local surfaces

$$k(\mathbf{x}, \mathbf{z}) = \sigma^2 \exp\left(-\frac{1}{2} \langle (\mathbf{x} - \mathbf{z}), \Sigma(\mathbf{x}, \mathbf{z})^{-1} (\mathbf{x} - \mathbf{z}) \rangle\right),$$

$$\Sigma(\mathbf{x}; \bar{X}) = \frac{1}{n-1} \sum_{\mathbf{y} \in \mathcal{N}_{\bar{X}}(\mathbf{x})} (\mathbf{y} - \mathbf{x})(\mathbf{y} - \mathbf{x})^\top, \quad (13a)$$

$$\Sigma(\mathbf{x}, \mathbf{z}) = \Sigma(\mathbf{x}; \bar{X}) + \mathbf{R}^\top \Sigma(\mathbf{z}; \bar{Z}) \mathbf{R}, \quad (13b)$$

where $\mathcal{N}_{\bar{X}}(\mathbf{x})$ are the n nearest points in $\bar{X}$ around $\mathbf{x}$, and $\mathbf{R}$ is the rotation matrix between X and Z . In the following, we let $\Sigma_{ij} \triangleq \Sigma(\mathbf{x}_i, \mathbf{z}_j)$, and (i) do a KD-tree ball-based lookup at $\mathbf{x}$, (ii) compute the distances to each point in this ball, and (iii) select the nearest n points to constitute this set. This approach generalizes the separate edge and surface loss functions in some ICP-based methods without dedicated feature extractions [47, 63], as shown in Fig. 2.

Remark 1 When the empirical covariance is used in (13), the induced anisotropy can render the loss function both sparse and noisy: the kernel attenuates components aligned with high-variance normal directions, which can be detrimental to registration performance. To regularize this degeneracy, we impose upper bounds on the dominant eigenmodes of $\Sigma(\mathbf{x}, \mathbf{z})$, constraining the normal-tangential

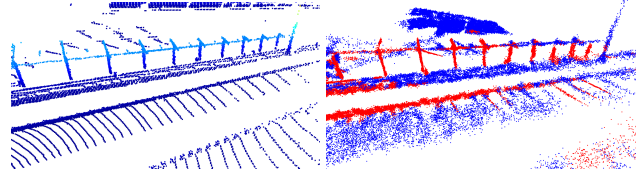


Figure 2. Raw point cloud Z (left), and resampled point cloud (right) in red if $\Sigma(\mathbf{z}_i; \bar{Z})$ has two small eigenvalues (edge) and blue if $\Sigma(\mathbf{z}_i; \bar{Z})$ has one small eigenvalue (surface).

weighting ratio and ensuring a numerically stable contribution of all geometric directions to the loss.

3.2. Calculating Riemannian gradients

In a first variant of G-CVO, we follow [23] and solve (11) by gradient ascent (hereinafter G-CVO-1). As the objective is a sum of exponentials on $\mathcal{G}$, taking gradients in a Euclidean space disregards the structure of $\mathcal{G}$ and can result in suboptimal linearizations [26]. Instead, we take the directional derivative of (12) at $\mathbf{T}$ along a perturbation $\xi^\wedge$ by (3) to derive the Riemannian gradients with the metric (2).

Lemma 1 The Riemannian gradient of (12) at $\mathbf{T} \in \mathcal{G}$ is $\text{grad} f(\mathbf{T})$ with $(\text{grad} f(\mathbf{T}))^\vee \triangleq \mathbf{g}(\mathbf{T})$, where $\mathbf{g}(\mathbf{T}) = (g_\omega^\top, g_v^\top)^\top \in \mathbb{R}^6$ is $\text{grad} f(\mathbf{T})$'s vector representation, $\mathbf{r}_{ij} = \mathbf{x}_i - \mathbf{T}^{-1} \mathbf{z}_j \in \mathbb{R}^3$,

$$g_v = - \sum_{ij} c_{ij} k(\mathbf{x}_i, \mathbf{T}^{-1} \mathbf{z}_j) \Sigma_{ij}^{-1} \mathbf{r}_{ij} := \sum_{ij} g_v^{(ij)}, \quad (14)$$

$$g_\omega = - \sum_{ij} c_{ij} k(\mathbf{x}_i, \mathbf{T}^{-1} \mathbf{z}_j) (\mathbf{T}^{-1} \mathbf{z}_j)^\wedge \Sigma_{ij}^{-1} \mathbf{r}_{ij} := \sum_{ij} g_\omega^{(ij)}.$$

Proof: The proof is in the supplementary materials, and follows from defining $\gamma(t) = \mathbf{T} \exp(t\xi^\wedge)$, evaluating $Df(\gamma(t))[\xi^\wedge]$, and using (3) to identify the gradients. $\square$

3.3. A first-order method

With Lemma 1, we can implement first-order optimizers to solve (11) with the anisotropic kernels in (13). To perform an update in the ascent direction, we calculate the step size μ using a line search along the lines of [16]. Specifically, we use a fourth-order expansion of (12) in the direction $\xi^\wedge$:

$$G(t) \triangleq f(\mathbf{T} \exp(t\xi^\wedge)) \approx \sum_{ij} c_{ij} k_{ij} \sum_{k=0}^4 g_{ij}^k t^k, \quad (15)$$

with the factors g_{ij}^k in the supplementary materials. We find the maximum value of this polynomial near 0 by computing the roots of $G(t)'$ in t . The smallest positive real root is chosen. If none are found, a minimum step $\mu_0 > 0$ is used

$$t^* = \min\{t \in \mathbb{C} \mid G'(t) = 0, \text{imag}(t) = 0\}, \quad (16a)$$

$$\mu = \max\{\mu_0, t^*\}. \quad (16b)$$

Algorithm 1 Generalized CVO (G-CVO).

```

1: receive  $\mathbf{T}^{(0)}, \mu_0, k_{\max}, \epsilon_{\text{tol}}, X, Z$ 
2: while  $\|\xi^{(k)}\|_1 \geq \epsilon_{\text{tol}}$  and  $k < k_{\max}$  do
3:   Transform  $Z$  with  $(\mathbf{T}^{(k)})^{-1}$ 
4:   Compute  $\{\Sigma_{ij}\}$  with  $\mathbf{R}$  from  $\mathbf{T}^{(k)}$  (13)
5:   Evaluate  $f$  at  $\mathbf{T}^{(k)}$  using  $\{\Sigma_{ij}\}_{ij}$  (12)
6:   Evaluate  $\text{grad}f(\mathbf{T}^{(k)})$  (14)
7:   if using 1st-order update then
8:     Evaluate  $\mu^{(k)}$  using a line-search (16)
9:      $\xi^{(k)} = \mu^{(k)} \text{grad}f(\mathbf{T}^{(k)})^\vee$  (17)
10:  else
11:    Evaluate  $\text{Hess}_{\text{GN}}f(\mathbf{T}^{(k)})$  (20)
12:     $\xi^{(k)} = -\text{Hess}_{\text{GN}}f(\mathbf{T}^{(k)})^{-1} \text{grad}f(\mathbf{T}^{(k)})^\vee$  (22)
13:  end if
14:  Update  $\mathbf{T}^{(k+1)} \leftarrow \mathbf{T}^{(k)} \exp([\xi^{(k)}]^\wedge)$ 
15:   $k \leftarrow k + 1$ 
16: end while
17: return  $\mathbf{T}^{(k)}$ 

```

The Riemannian gradient ascent step is implemented as

$$\mathbf{T}^{(k+1)} = \mathbf{T}^{(k)} \exp(\mu^{(k)} \text{grad}f(\mathbf{T}^{(k)})), \quad (17)$$

where $\mu^{(k)}$ is computed by (16) at $\mathbf{T}^{(k)}$.

3.4. A second-order method

The first-order solver can be slow, as shown in Sec. 4.4, when the same step size is applied to all dimensions of $\mathbf{T}$ without using curvature information. As (12) is observed locally quadratic on $\mathcal{G}$ (see Sec. C), we propose a Riemannian Gauss-Newton (GN) method using an approximated Riemannian Hessian. In each iteration, we first transform Z with the current estimate as $\mathbf{T}^{-1}Z$. Then we denote

$$f_{ij} \triangleq c_{ij}k(\mathbf{x}_i, \mathbf{T}^{-1}\mathbf{z}_j), \quad J_j \triangleq [-(\mathbf{T}^{-1}\mathbf{z}_j)^\wedge \mathbf{I}] \in \mathbb{R}^{3 \times 6},$$

$$\mathbf{D}(\mathbf{r}_{ij})[\xi] = J_j \xi \quad (18)$$

By (8), it suffices to derive $\mathbf{D}(\mathbf{g}(\mathbf{T}))[\xi]$ in left-trivialized coordinates and then append the connection term from (5).

$$\mathbf{D}(\mathbf{g}(\mathbf{T}))[\xi] = -\left(\sum_{ij} J_j^\top \Sigma_{ij}^{-1} \mathbf{r}_{ij} \mathbf{D}(f_{ij})[\xi] + \right. \quad (19a)$$

$$f_{ij} \mathbf{D}(J_j^\top)[\xi] \Sigma_{ij}^{-1} \mathbf{r}_{ij} + f_{ij} J_j^\top \mathbf{D}(\Sigma_{ij}^{-1})[\xi] \mathbf{r}_{ij} \quad (19b)$$

$$\left. + f_{ij} J_j^\top \Sigma_{ij}^{-1} \mathbf{D}(\mathbf{r}_{ij})[\xi]\right). \quad (19c)$$

We approximate this directional derivative by the dominant last term in (19c), as discussed in Remark 2, resulting in

$$(\text{Hess}f(\mathbf{T})[\xi^\wedge])^\vee \approx (\text{Hess}_{\text{GN}}f(\mathbf{T})[\xi^\wedge])^\vee \quad (20a)$$

$$:= -\sum_{ij} f_{ij} J_j^\top \Sigma_{ij}^{-1} J_j \xi + \Gamma^\vee(\xi^\wedge, \text{grad}f(\mathbf{T})), \quad (20b)$$

where, by Lemma 1 and (6), the correction term is

$$\Gamma^\vee(\xi^\wedge, \text{grad}f(\mathbf{T})) = \begin{bmatrix} -\frac{1}{2}g_\omega^\wedge & 0 \\ -g_v^\wedge & 0 \end{bmatrix} \xi. \quad (21)$$

As we are maximizing the object, the step update of the second-order solver is implemented as

$$\mathbf{T}^{(k+1)} = \mathbf{T}^{(k)} \exp([-(\text{Hess}_{\text{GN}}f(\mathbf{T}^{(k)}))^{-1} \text{grad}f(\mathbf{T}^{(k)})^\vee]^\wedge). \quad (22)$$

We refer to (22) as G-CVO-2, as summarized in Alg. 1.

Remark 2 In the implementation of G-CVO-2, the covariances Σ_{ij} are updated once per iteration and treated as constants within each step update. We further retain only the last term in (19c). This inexact Hessian approximation is motivated by inexact GN schemes [24] and supported by the KITTI ablation results in Sec. 4.4: using the exact Hessian in (22) provides marginal gains in convergence speed while increasing per-iteration cost, resulting in higher overall compute time for solving (11).

Remark 3 By instead letting $\Sigma(\mathbf{x}, \mathbf{z}) \equiv \mathbf{I}$ for all $\mathbf{x}, \mathbf{z} \in \mathbb{R}^3$, we obtain the kernel formulations of CVO [16] and SemanticCVO [64]. As such, the exact and approximate GN method of G-CVO can be applied in these settings as well.

Remark 4 In general, $\text{grad}f$ is not a left-invariant vector field for a function f defined on $\mathcal{G}$. However, in Alg. 1, we re-express the problem at each iteration by transforming Z with the inverse of the previous iterate, $(\mathbf{T}^{(k)})^{-1}$. Consequently, each registration step is performed from the identity, aligning with $(\mathbf{T}^{(k)})^{-1}Z$. This approach avoids transporting the gradients from $T_{\mathbf{T}^{(k)}}\mathcal{G}$ back to $\mathfrak{g}$.

4. Experiments

We implement G-CVO in CUDA and evaluate all methods on a computer equipped with an RTX 4090 GPU and Intel Xeon CPU at 2.70GHz. We evaluate the proposed G-CVO method on several public and private datasets, designed to benchmark the method on different sensor modalities with varying degrees of feature-sparsity and outliers. We primarily consider frame-to-frame tracking in consecutive frame pairs rather than a full visual odometry pipeline, and evaluate G-CVO in feature-rich urban driving scenarios in Sec. 4.1 and more challenging feature-sparse vehicle racing environments in Sec. 4.2. In addition, we evaluate tracking performance in an indoor RGB-D dataset in Sec. 4.3. We study convergence and compute times in Sec. 4.4, respectively, and present object registration results in Sec. 4.5.

4.1. LiDAR Tracking in Feature-rich Scenes

First, we consider frame-to-frame registration for sequential frame pairs in the KITTI LiDAR dataset [22]. This dataset

Table 1. Registration errors (translation and rotation) for different methods on standard KITTI sequences.

Sequence	G-CVO-2 (Ours)		G-CVO-1 (Ours)		Fast-VGICP [32]		NDT [37]		GICP [46]		ICP [5]		CVO [23, 64] (1st-order)		GeoTransformer [42]	
	Trans.	Rot.	Trans.	Rot.	Trans.	Rot.	Trans.	Rot.	Trans.	Rot.	Trans.	Rot.	Trans.	Rot.	Trans.	Rot.
00	1.0649	0.0058	1.0060	0.0057	1.1653	0.0055	57.6272	0.2386	28.2713	0.0249	6.7379	0.0291	1.8087	0.0128	-	-
01	2.0752	0.0066	1.9665	0.0065	8.5028	0.0080	94.7583	0.0928	94.1168	0.0825	21.3254	0.0158	4.3708	0.0148	-	-
02	1.4174	0.0062	1.4452	0.0072	1.6059	0.0055	68.3716	0.2149	34.4085	0.0483	9.5617	0.0307	1.6152	0.0087	-	-
03	1.5743	0.0087	2.7747	0.0179	1.1627	0.0073	87.5822	0.1238	1.7717	0.0058	10.3611	0.0449	2.1206	0.0120	-	-
04	0.4718	0.0070	0.4296	0.0050	1.3032	0.0078	97.1345	0.0083	100.3140	0.0034	9.3629	0.0404	1.2642	0.0152	-	-
05	1.0557	0.0060	2.1047	0.0116	1.0113	0.0053	54.8189	0.2458	27.6153	0.0092	3.4138	0.0160	1.1743	0.0075	-	-
06	1.1641	0.0070	0.7694	0.0066	0.8082	0.0049	59.0391	0.2848	29.2058	0.0036	4.7885	0.0192	0.8103	0.0079	-	-
07	1.1106	0.0065	0.5777	0.0045	0.7967	0.0052	57.6573	0.3708	33.9593	0.0051	4.0295	0.0262	1.0623	0.0095	-	-
08	1.3356	0.0055	1.5383	0.0068	1.2516	0.0053	57.6909	0.2112	24.6989	0.0051	9.6332	0.0448	1.9684	0.0103	6.8211	0.0189
09	2.3890	0.0091	0.9627	0.0047	1.1934	0.0052	71.9249	0.2264	25.7434	0.0251	8.5134	0.0308	3.6727	0.0164	8.7523	0.0216
10	1.6250	0.0076	1.7445	0.0089	1.3606	0.0055	78.7093	0.2092	12.4857	0.0070	10.2373	0.0377	1.8504	0.0128	10.8313	0.0320
Mean	1.3894	0.0069	1.3927	0.0078	1.8329	0.0060	71.3922	0.2024	37.5082	0.0200	8.9059	0.0305	1.9744	0.0116	-	-
Std.	0.5244	0.0012	0.7217	0.0039	2.2245	0.0011	15.8960	0.0975	31.0308	0.0250	4.8482	0.0107	1.1024	0.0031	-	-

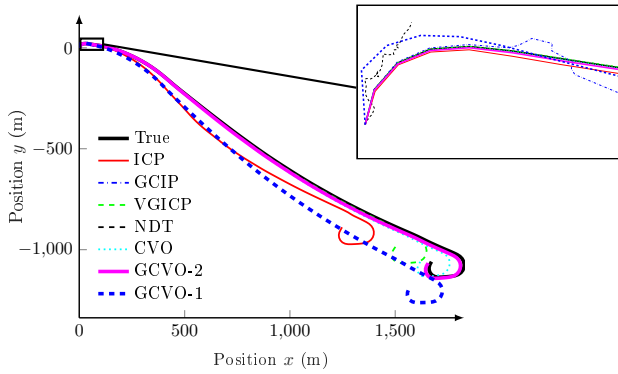


Figure 3. G-CVO versus baselines on KITTI sequence 01.

contains 11 LiDAR sequences with ground truth poses in urban driving scenarios. Each frame is downsampled with a voxel filter of 0.25m. As evaluation metrics, we consider KITTI odometry benchmark’s official Translational Error and Rotational Error, computed over various subsequence lengths (100m, 200m, 400m, 800m). We compare G-CVO to various ICP baselines, including ICP [5], GICP [46] implemented with PCL [43], Fast-VGICP [32], and a GMM-based method NDT [37]. We also include the classical CVO [16, 64] as a correspondence-free baseline. We include GeoTransformer [42], a learning-based method with strong global registration performance, but only report its test sequences 08–10 (training sequences are omitted).

Quantitative results are summarized in Table 1, with qualitative results on sequence 01 in Fig. 3. G-CVO reduces translation errors and is on par with Fast-VGICP with respect to rotational errors. In particular, for the less feature-dense highway sequences 01 and 04 where correspondences are difficult to find, the proposed method outperforms the baselines by a substantial margin.

4.2. LiDAR Tracking in Feature-sparse Scenes

To evaluate the robustness of G-CVO in high-speed, feature-sparse environments, we collect a dataset with a Velodyne 128-beam LiDAR at a closed-course race track. The dataset consists of three sequences with ground truth

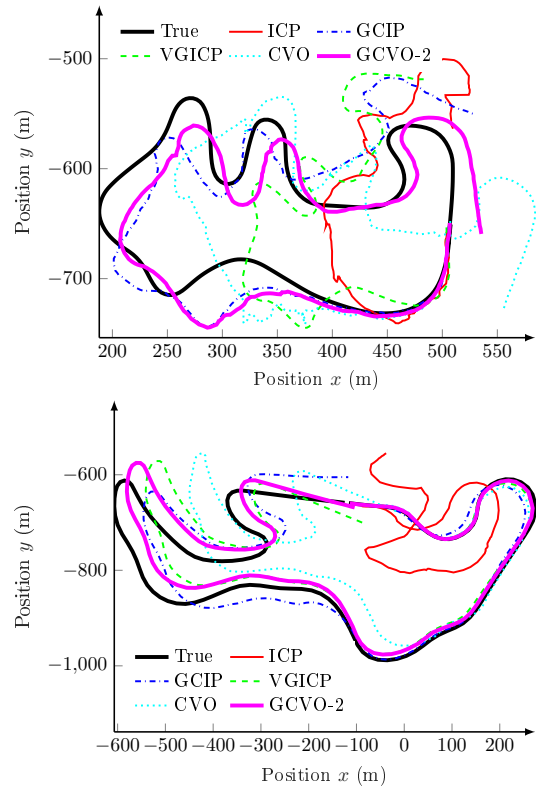


Figure 4. Ground truth and frame-to-frame tracking with G-CVO-2 on the racing dataset (Dirt Track top, Race track bot.).

poses from the RTK GPS system [40], including one sequence on a large empty skid pad (Skid Pad), one sequence on a 2-mile paved racetrack (Race Track), and one sequence on a dirt track (Dirt Track). Images from an on-board camera and sample point clouds are provided in the supplementary materials. To achieve convergence times below 100ms, we downsample each frame using a voxel size of 2m. As an additional baseline, we run the classical CVO with isotropic kernels using the proposed second-order solver (see Remark 3). All the solvers are limited to a maximum of 200 iterations, mirroring typical LiDAR odometry settings [57]. We use the same standard KITTI

Table 2. Registration errors (translation and rotation) for different methods on the three feature-sparse racing sequences.

Sequence	G-CVO-2 (Ours)		G-CVO-1 (Ours)		Fast-VGICP [32]		GICP [46]		ICP [5]		CVO (2nd-order)	
	Trans.	Rot.	Trans.	Rot.	Trans.	Rot.	Trans.	Rot.	Trans.	Rot.	Trans.	Rot.
Skid Pad	2.5076	0.0149	7.1536	0.0404	1.8999	0.0138	29.9657	0.4065	30.4623	0.0917	4.2126	0.0270
Dirt Track	6.0791	0.0167	20.8969	0.0772	26.3557	0.0807	9.7605	0.0557	33.1731	0.1404	15.0181	0.0492
Race Track	4.3702	0.0101	31.5965	0.0479	6.3208	0.0133	8.4150	0.0193	40.1477	0.0667	10.4601	0.0183
Mean	4.3189	0.0139	19.8823	0.0552	11.5255	0.0359	16.0471	0.1605	34.5944	0.0996	9.8969	0.0315
Std.	1.7863	0.0034	12.2530	0.0195	13.0322	0.0388	12.0727	0.2138	4.9967	0.0375	5.4247	0.0159

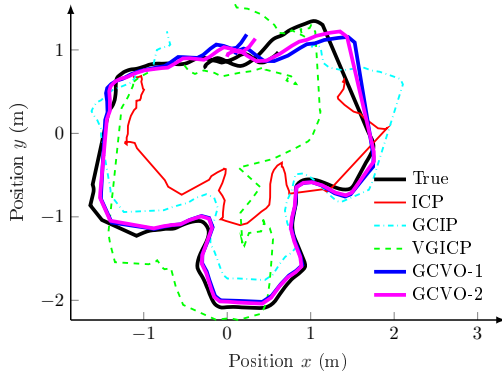


Figure 5. Ground truth (black) and frame-to-frame tracking using baselines and G-CVO-1 and G-CVO-2 on ETH3D (table_3).

metrics as Sec. 4.1. Quantitative results are reported in Table 2, using a subset of the baselines in Sec. 4.1.

In the Skid Pad sequence, few features fall within the LiDAR’s effective range, leaving the registration vulnerable to incorrect matches [5, 46]. The Race Track represents a different challenge due to high-speed driving, leading to large movements between consecutive frames. Furthermore, the sequence covers long distances (2 miles), making frame-to-frame registrations prone to long-term drifts. Finally, the Dirt Track poses a unique challenge due to the high vibration of the chassis and significant dust clouds that emerge behind the car. Despite these challenges, G-CVO-2 demonstrates improved robustness in these scenes compared to baselines. Compared with surface-informed techniques (G-CVO, GICP) general perform better than methods using point-wise residuals like ICP and classical CVO perform, even when CVO is implemented with a second-order solver.

4.3. RGB-D Tracking in Indoor Scenes

Next, we perform frame-to-frame registration on the indoor dataset with RGB-D point clouds, ETH3D [45], on the following five sequences: `sfm_bench`, `plant_1`, `table_3`, `sfm_lab_room_1`, `planar_2`. We downsample each frame to around 3000 points. We assess tracking accuracy using the RMSE of Relative Pose Error (RPE) and Absolute Pose Error (APE) over the estimated trajectories with `evo` [27]. RPE (with $\Delta = 1$) quantifies local drift in relative motion (position and orientation), while APE measures the global drift in absolute position against ground truth.

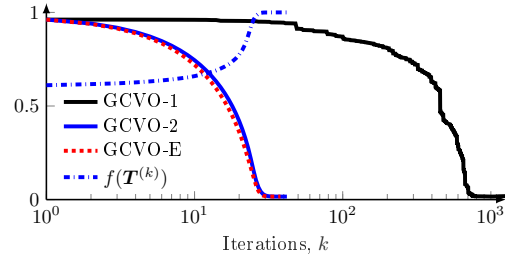


Figure 6. Normalized Frobenius error with the 1st/2nd-order optimizers in GCVO (black/blue/red) over the iterates on an input pair of KITTI sequence 03. For reference, we report the inner-product objective of in GCVO-2 (blue, dashed).

Quantitative results are shown in Table 3, with qualitative results on table_3 in Fig. 5. G-CVO has the best tracking accuracy in three sequences, and has the second-best performance for the other two. On average, G-CVO outperforms all baselines in terms of APE, and is the second-best method in terms of RPE.

4.4. Computational Properties and Approximations

To provide intuition, we study the convergence of G-CVO on the KITTI LiDAR dataset. We compare the number of iterations and total time to converge in Fig. 6 and Fig. 7. We compare G-CVO-1, G-CVO-2, and a version of G-CVO that implements the exact Hessian, referred to as G-CVO-E.

First, we consider the frame pair of indices 000000 and 000001 from KITTI sequence 03, which has a ground truth relative pose T_{gt} with a translation of approximately 0.95 (m). We report the change of the Frobenius error $\|T_{gt}^{-1}T^{(k)} - I\|_F$ with respect to the iterates k in Fig. 6. G-CVO-2 converges in only ~ 30 iterations, and G-CVO-1 requires more than 1200 iterations to reach the same tolerances. The increase of the inner product objective $f(T)$ correlates well with the decrease in error.

In Fig. 7, we compare the convergence time, excluding IO preprocessing, averaged over a sequence of 100 frames from the KITTI sequence 04, as a function of the number of points in the registration (N_X and N_Z). The methods are evaluated both without an initial guess ($T^{(0)} = I$) and with an initial guess (using a constant velocity assumption to initialize $T^{(0)}$), as is typically done in visual and LiDAR odometry [57]. GCVO-2 converges in less than 100ms with around 10^4 points in the frame-to-frame track-

Table 3. Tracking errors on the five ETH3D RGB-D sequences: RPE RMSE ($\Delta=1$) and APE RMSE for each method.

Sequence	G-CVO-2 (Ours)		G-CVO-1 (Ours)		Fast-VGICP [32]		GICP [46]		ICP [5]		GeoTransformer [42]	
	RPE RMSE	APE RMSE	RPE RMSE	APE RMSE	RPE RMSE	APE RMSE	RPE RMSE	APE RMSE	RPE RMSE	APE RMSE	RPE RMSE	APE RMSE
sfm_bench	0.0157	0.3426	0.1730	7.1005	0.0178	1.0861	0.0058	0.3568	0.0259	1.2655	0.0217	1.2327
plant_1	0.0021	0.0095	0.0022	0.0162	0.0217	0.1842	0.0034	0.0252	0.0086	0.1279	0.3572	2.2901
table_3	0.0046	0.1722	0.0054	0.2293	0.0429	0.5669	0.0145	0.6896	0.0269	1.0142	-	-
sfm_lab_room_1	0.0187	0.3712	0.0332	0.3315	0.0189	0.2522	0.0041	0.0304	0.0232	0.3753	0.0210	0.3418
planar_2	0.0021	0.0610	0.0021	0.0540	0.0177	0.3883	0.0044	0.1429	0.0103	0.4007	0.0099	0.3638
Mean	0.0086	0.1913	0.0432	1.5463	0.0238	0.4956	0.0064	0.2490	0.0190	0.6367	-	-
Std.	0.0079	0.1625	0.0737	3.1075	0.0108	0.3611	0.0046	0.2806	0.0088	0.4798	-	-

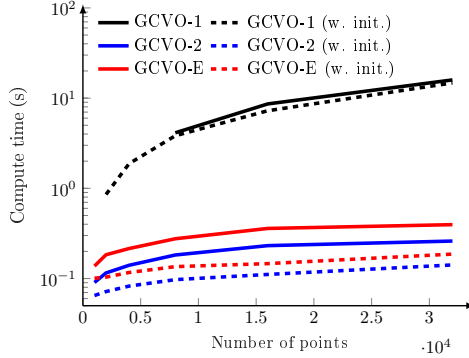


Figure 7. Average convergence time (excluding IO preprocessing) for G-CVO-1/2/E on KITTI sequence 04, with and without warm-starting the optimizer using a constant velocity assumption. ing of Sec. 4.2. GCVO-1 is one order of magnitude slower in this setting. While G-CVO-E and G-CVO-2 converge to the same solution, the approximation in G-CVO-2 decreases compute time by $\sim 50\%$ compared to G-CVO-E irrespective of the quality of the initial guess.

The per-iteration complexity of G-CVO is $O(N_X N_Z)$, compared to $O(N_X \log N_Z)$ for KD-tree-based ICP. VG-ICP [32] achieves additional speedups via voxel hashing, which is not used in G-CVO. To evaluate runtime scaling, we report time-accuracy trade-offs on seven KITTI sequences (01–07) with $N \in 4, 8, 16$ k voxel-downsampled points (Fig. 8). G-CVO and VGICP-GPU run on GPU, while others use CPU. Despite higher per-iteration cost, G-CVO attains higher accuracy than VGICP (Fig. 8) and converges faster than CVO’s first-order solver (Fig. 6) due to second-order optimization. Voxel downsampling takes 0.0413 s per frame; combined with convergence time, G-CVO achieves 10 Hz tracking with 4k points.

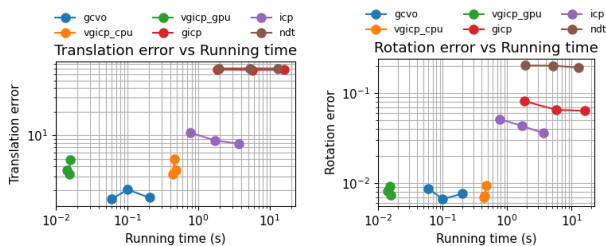


Figure 8. Convergence time (IO/downsampling excluded) vs accuracy. Accuracy in KITTI metrics at $N = \{4, 8, 16\} \cdot 10^3$ points.

Table 4. Mean object registration errors on ModelNet40 [55]

Method (*: init with [42])	$\sigma^2 = 0.01$		$\sigma^2 = 0.01, \gamma = 0.2$		$\sigma^2 = 0.01, \text{crop}=30\%$	
	rot. (deg)	trans. (m)	rot. (deg)	trans. (m)	rot. (deg)	trans. (m)
RANSAC-50k	105.9665	0.5936	104.2570	0.5871	111.6881	0.6616
VGICP	38.6575	0.4484	43.6178	0.4832	41.6356	0.4643
GeoTransformer [42]	0.6495	0.0088	0.9341	0.0110	4.2401	0.0460
GCVO-2 (Ours)	0.2669	0.0024	3.0607	0.0206	32.7753	0.2829
VGICP*	-	-	1.0363	0.0103	6.0881	0.1229
GCVO-2* (Ours)	-	-	0.3549	0.0014	3.8465	0.0401

4.5. Object Registration

While the primary use of G-CVO is in fine-grained local registration for tracking, we also evaluate its object-level registration capability on ModelNet40 [55] (Tab. 4). Each model is randomly transformed with mean rotation and translation of 45° and 0.5m, with maximum magnitudes of 80° and 0.85m. We further apply zero-mean Gaussian noise ($\sigma^2 = 0.01$), uniformly-distributed outliers ($\gamma = 0.2$), and randomly crop 30% of the points using a plane in both point clouds. We compare against VGICP, RANSAC (50k iterations), and GeoTransformer [42], a learning-based global registration method. G-CVO consistently outperforms baselines that are not learning-based. Without cropping or outliers, it surpasses GeoTransformer; under reduced overlap, performance degrades due to the absence of learned invariant features. Refining GeoTransformer outputs with G-CVO consistently improves pose accuracy.

5. Conclusions

We propose a correspondence-free, local-geometry-aware registration method to align point clouds by maximizing the inner product between their functional representations in an RKHS. In tracking, G-CVO achieves lower drift than CVO and ICP-based baselines on public and self-collected LiDAR/RGB-D datasets. It can also be used for local refinement in global registration tasks. The use of anisotropic kernels encourages surface-aware alignment, while the second-order Riemannian optimization enables fast convergence.

A limitation of the approach is relying on a GPU for fast evaluations of dense pairwise kernel functions and Jacobians. Additionally, the proposed solvers target odometry applications that assume sufficient overlap between the two inputs, which may not be respected in large motion or low-overlap registrations. Future works include integrating the registration G-CVO in complete visual and LiDAR odometry systems as well as extending the objective function into differentiable layers of deep learning frameworks.

References

- [1] P-A Absil, Robert Mahony, and Rodolphe Sepulchre. *Optimization algorithms on matrix manifolds*. Princeton University Press, 2008.
- [2] Khurruam Aftab and Richard Hartley. Convergence of iteratively re-weighted least squares to robust M-estimators. In *2015 IEEE Winter Conference on Applications of Computer Vision*, pages 480–487, 2015.
- [3] Andrea Azzini et al. Balo: A novel point-to-plane balanced lidar odometry. In *Proceedings of the IEEE Intelligent Vehicles Symposium (IV)*, 2025.
- [4] Jens Behley and Cyrill Stachniss. Efficient surfel-based SLAM using 3D laser range data in urban environments. In *Robotics: science and systems*, page 59, 2018.
- [5] Paul J Besl and Neil D McKay. Method for registration of 3D shapes. In *Sensor fusion IV: control paradigms and data structures*, pages 586–606. Spie, 1992.
- [6] Uttaran Bhattacharya and Venu Madhav Govindu. Efficient and robust registration on the 3D special Euclidean group. In *Int. Conf. on Computer Vision*, pages 5885–5894, 2019.
- [7] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*. Springer, 2006.
- [8] Nicolas Boumal. *An introduction to optimization on smooth manifolds*. Cambridge University Press, 2023.
- [9] Yohan Breux, André Mas, and Lionel Lapierre. On-manifold probabilistic iterative closest point: Application to underwater karst exploration. *Int. Journal of Robotics Research*, 41 (9-10):875–902, 2022.
- [10] Cesar Cadena, Luca Carlone, Henry Carrillo, Yasir Latif, Davide Scaramuzza, Jose Neira, Ian Reid, John J Leonard, et al. Past, present, and future of simultaneous localization and mapping: Towards the robust-perception age. *IEEE Transactions on robotics*, 32(6):1309–1332, 2016.
- [11] Yixi Cai, Wei Xu, and Fu Zhang. IKD-tree: An incremental KD-tree for robotic applications. *arXiv preprint arXiv:2102.10808*, 2021.
- [12] Andrea Censi. An ICP variant using a point-to-line metric. In *Int. Conf. on robotics and automation*, pages 19–25. Ieee, 2008.
- [13] Yang Chen and Gérard Medioni. Object modelling by registration of multiple range images. *Image and vision computing*, 10(3):145–155, 1992.
- [14] Zhi Chen, Kun Sun, Fan Yang, and Wenbing Tao. Sc2-pcr: A second order spatial compatibility for efficient and robust point cloud registration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 13221–13231, 2022.
- [15] Christopher Choy, Jaesik Park, and Vladlen Koltun. Fully convolutional geometric features. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 8958–8966, 2019.
- [16] William Clark, Maani Ghaffari, and Anthony Bloch. Non-parametric continuous sensor registration. *Journal of Machine Learning Research*, 22(271):1–50, 2021.
- [17] Arun Das and Steven L Waslander. Scan registration using segmented region growing NDT. *Int. journal of Robotics Research*, 33(13):1645–1663, 2014.
- [18] Jean-Emmanuel Deschaud. IMLS-SLAM: scan-to-model matching based on 3D data. In *Int. conf. on robotics and aut.*, pages 2480–2485. IEEE, 2018.
- [19] Franck Djeumou, Thomas Lew, Nan Ding, Michael Thompson, Makoto Suminaka, Marcus Greiff, and John Subosits. One model to drift them all: Physics-informed conditional diffusion model for driving at the limits. In *Conf. on Robot Learning*, 2024.
- [20] Benjamin Eckart, Kihwan Kim, and Jan Kautz. HGMR: Hierarchical Gaussian mixtures for adaptive 3D registration. In *European conf. on computer vision*, pages 705–721, 2018.
- [21] JEAN QUAINANCE Gallier and Jocelyn Quaintance. *Differential geometry and lie groups*. Springer, 2020.
- [22] Andreas Geiger, Philip Lenz, and Raquel Urtasun. Are we ready for autonomous driving? the KITTI vision benchmark suite. In *Conf. on Computer Vision and Pattern Recognition*, 2012.
- [23] Maani Ghaffari, William Clark, Anthony Bloch, Ryan M Eustice, and Jessie W Grizzle. Continuous direct sparse visual odometry from RGB-D images. *Robotics: Science and Systems*, 2019.
- [24] Serge Gratton, Amos S Lawless, and Nancy K Nichols. Approximate Gauss-Newton methods for nonlinear least squares problems. *SIAM Journal on Optimization*, 18(1): 106–132, 2007.
- [25] Michael Greenspan and Mike Yurick. Approximate KD tree search for efficient ICP. In *Int. conf. on 3D Digital Imaging and Modeling*, pages 442–448. IEEE, 2003.
- [26] Giorgio Grisetti, Rainer Kümmerle, Cyrill Stachniss, Udo Frese, and Christoph Hertzberg. Hierarchical optimization on manifolds for online 2d and 3d mapping. In *International Conference on Robotics and Automation*, pages 273–278, 2010.
- [27] Michael Grupp. evo: Python package for the evaluation of odometry and SLAM. <https://github.com/MichaelGrupp/evo>, 2017.
- [28] Shengyu Huang, Zan Gojcic, Mikhail Usvyatsov, Andreas Wieser, and Konrad Schindler. Predator: Registration of 3d point clouds with low overlap. In *Proceedings of the IEEE/CVF Conference on computer vision and pattern recognition*, pages 4267–4276, 2021.
- [29] Peter J Huber. Robust estimation of a location parameter. In *Breakthroughs in statistics: Methodology and distribution*, pages 492–518. Springer, 1992.
- [30] Yunho Kim, Chanyoung Kim, and Jemin Hwangbo. Learning forward dynamics model and informed trajectory sampler for safe quadruped navigation. In *Robotics: Science and Systems (RSS)*, 2022.
- [31] Kenji Koide, Masashi Yokozuka, Shuji Oishi, and Atsuhiko Banno. Globally consistent 3D LiDAR mapping with GPU-accelerated GICP matching cost factors. *Robotics and Aut. Letters*, 6(4):8591–8598, 2021.
- [32] Kenji Koide, Masashi Yokozuka, Shuji Oishi, and Atsuhiko Banno. Voxelized GICP for fast and accurate 3D point cloud registration. In *Int. conf. on robotics and automation*, pages 11054–11059. IEEE, 2021.

- [33] Dongjae Lee, Minwoo Jung, Woosong Yang, and Ayoun Kim. LiDAR odometry survey: recent advancements and remaining challenges. *Intelligent Service Robotics*, 17(2): 95–118, 2024.
- [34] Thomas Lew, Marcus Greiff, Franck Djeumou, Makoto Suminaka, Michael Thompson, and John Subosits. Risk-averse model predictive control for racing in adverse conditions. *Int. Conf. on Robotics and Automation*, 2025.
- [35] Yin Li, Manohar Paluri, James M Rehg, and Piotr Dollár. Unsupervised learning of edges. In *Conf. on computer vision and pattern recognition*, pages 1619–1627, 2016.
- [36] Chien Erh Lin, Minghan Zhu, and Maani Ghaffari. Se3et: Se(3)-equivariant transformer for low-overlap point cloud registration. *IEEE Robotics and Automation Letters*, 2024.
- [37] Martin Magnusson. *The three-dimensional normal-distributions transform: an efficient representation for registration, surface analysis, and loop detection*. PhD thesis, Örebro university, 2009.
- [38] Yanzi Miao, Yang Liu, Hongbin Ma, and Huijie Jin. The pose estimation of mobile robot based on improved point cloud registration. *Int. Journal of Advanced Robotic Systems*, 13(2):52, 2016.
- [39] Martin Oelsch, Mojtaba Karimi, and Eckehard Steinbach. R-LOAM: Improving LiDAR odometry and mapping with point-to-mesh features of a known 3D reference object. *Robotics and Automation Letters*, 6(2):2068–2075, 2021.
- [40] Oxts. RT3000 Product Page, 2024. Last accessed 05/20/2024.
- [41] Anh Viet Phan, Minh Le Nguyen, Yen Lam Hoang Nguyen, and Lam Thu Bui. Dgcn: A convolutional neural network over large-scale labeled graphs. *Neural Networks*, 108:533–543, 2018.
- [42] Zheng Qin, Hao Yu, Changjian Wang, Yulan Guo, Yuxing Peng, Slobodan Ilic, Dewen Hu, and Kai Xu. Geotransformer: Fast and robust point cloud registration with geometric transformer. *Trans. on Pattern Analysis and Machine Intelligence*, 45(8):9806–9821, 2023.
- [43] Radu Bogdan Rusu and Steve Cousins. 3D is here: Point cloud library (PCL). In *Int. Conf. on robotics and automation*, pages 1–4. IEEE, 2011.
- [44] Radu Bogdan Rusu, Nico Blodow, and Michael Beetz. Fast point feature histograms (fpfh) for 3d registration. In *2009 IEEE international conference on robotics and automation*, pages 3212–3217. IEEE, 2009.
- [45] Thomas Schops, Torsten Sattler, and Marc Pollefeys. Bad SLAM: Bundle adjusted direct RGB-D SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 134–144, 2019.
- [46] Aleksandr Segal, Dirk Haehnel, and Sebastian Thrun. Generalized-ICP. In *Robotics: science and systems*, page 435. Seattle, WA, 2009.
- [47] Tixiao Shan and Brendan Englot. Lego-loam: Lightweight and ground-optimized LiDAR odometry and mapping on variable terrain. In *Int. conf. on intelligent robots and systems*, pages 4758–4765. IEEE, 2018.
- [48] Leonard A Stefanski and Dennis D Boos. The calculus of M-estimation. *The American Statistician*, 56(1):29–38, 2002.
- [49] Pei Sun, Henrik Kretzschmar, Xerxes Dotiwalla, Aurelien Chouard, Vijaysai Patnaik, Paul Tsui, James Guo, Yin Zhou, Yuning Chai, Benjamin Caine, et al. Scalability in perception for autonomous driving: Waymo open dataset. In *Conf. on computer vision and pattern recognition (CVPR)*, pages 2446–2454, 2020.
- [50] Yanghai Tsin and Takeo Kanade. A correlation-based approach to robust point set registration. In *European conf. on computer vision*, pages 558–569. Springer Berlin Heidelberg, 2004.
- [51] Ignacio Vizzo, Xieyuanli Chen, Nived Chebrolu, Jens Behley, and Cyrill Stachniss. Poisson surface reconstruction for LiDAR odometry and mapping. In *Int. conf. on robotics and automation (ICRA)*, pages 5624–5630. IEEE, 2021.
- [52] Ignacio Vizzo, Tiziano Guadagnino, Benedikt Mersch, Louis Wiesmann, Jens Behley, and Cyrill Stachniss. Kiss-ICP: In defense of point-to-point ICP—simple, accurate, and robust registration if done the right way. *Robotics and Automation Letters*, 8(2):1029–1036, 2023.
- [53] Han Wang, Chen Wang, Chun-Lin Chen, and Lihua Xie. F-loam: Fast LiDAR odometry and mapping. In *Inf. Conf. on Intelligent Robots and Systems*, pages 4390–4396. IEEE, 2021.
- [54] Christopher KI Williams and Carl Edward Rasmussen. *Gaussian processes for machine learning*. MIT press Cambridge, MA, 2006.
- [55] Zhirong Wu, Shuran Song, Aditya Khosla, Fisher Yu, Linguang Zhang, Xiaoou Tang, and Jianxiong Xiao. 3d shapenets: A deep representation for volumetric shapes. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1912–1920, 2015.
- [56] Wei Xu and Fu Zhang. FAST-LIO: A fast, robust LiDAR-inertial odometry package by tightly-coupled iterated kalman filter, 2021.
- [57] Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin, and Fu Zhang. FAST-LIO2: Fast direct LiDAR-inertial odometry. *Transactions on Robotics*, 38(4):2053–2073, 2022.
- [58] Heng Yang, Jingnan Shi, and Luca Carlone. Teaser: Fast and certifiable point cloud registration. *Trans. on Rob.*, 37(2):314–333, 2020.
- [59] Jiaolong Yang, Hongdong Li, Dylan Campbell, and Yunde Jia. Go-ICP: A globally optimal solution to 3D ICP point-set registration. *Trans. on pattern analysis and machine intelligence*, 38(11):2241–2254, 2015.
- [60] Jiaqi Yang, Xiyu Zhang, Peng Wang, Yulan Guo, Kun Sun, Qiao Wu, Shikun Zhang, and Yanning Zhang. Mac: Maximal cliques for 3d registration. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2024.
- [61] Hao Yu, Fu Li, Mahdi Saleh, Benjamin Busam, and Slobodan Ilic. Cofinet: Reliable coarse-to-fine correspondences for robust pointcloud registration. *Advances in Neural Information Processing Systems*, 34:23872–23884, 2021.
- [62] Andy Zeng, Shuran Song, Matthias Nießner, Matthew Fisher, Jianxiong Xiao, and Thomas Funkhouser. 3dmatch: Learning local geometric descriptors from RGB-D reconstructions. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1802–1811, 2017.

- [63] Ji Zhang, Sanjiv Singh, et al. LOAM: LiDAR odometry and mapping in real-time. In *Robotics: Science and systems*, pages 1–9. Berkeley, CA, 2014.
- [64] Ray Zhang, Tzu-Yuan Lin, Chien Erh Lin, Steven A Parkison, William Clark, Jessy W Grizzle, Ryan M Eustice, and Maani Ghaffari. A new framework for registration of semantic point clouds from stereo and RGB-D cameras. In *Int. Conf. on Robotics and Automation*, pages 12214–12221. IEEE, 2021.
- [65] Ray Zhang, Zheming Zhou, Min Sun, Omid Ghasemalizadeh, Cheng-Hao Kuo, Ryan M Eustice, Maani Ghaffari, and Arnie Sen. Correspondence-free SE(3) point cloud registration in RKHS via unsupervised equivariant learning. In *European conf. on computer vision*, pages 68–86. Springer, 2024.
- [66] Ray Zhang, Jingwei Song, Xiang Gao, Junzhe Wu, Tianyu Liu, Jinyuan Zhang, Ryan Eustice, and Maani Ghaffari. RKHS-BA: A robust correspondence-free multi-view bundle adjustment framework for semantic point clouds. *Trans. on Pattern Analysis and Machine Intelligence*, 2025.
- [67] Yongjun Zhang, Pengcheng Shi, and Jiayuan Li. LiDAR-based place recognition for autonomous driving: A survey. *ACM Computing Surveys*, 57(4):1–36, 2024.
- [68] Guiyu Zhao, Sheng Ao, Ye Zhang, Kai Xu, and Yulan Guo. Progressive correspondence regenerator for robust 3d registration. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 1210–1219, 2025.
- [69] Shibo Zhao, Yuanjun Gao, Tianhao Wu, Damanpreet Singh, Rushan Jiang, Haoxiang Sun, Mansi Sarawata, Yuheng Qiu, Warren Whittaker, Ian Higgins, Yi Du, Shaoshu Su, Can Xu, John Keller, Jay Karhade, Lucas Nogueira, Sourojit Saha, Ji Zhang, Wenshan Wang, Chen Wang, and Sebastian Scherer. SubT-MRS Dataset: Pushing SLAM towards all-weather environments. In *Conf. on Computer Vision and Pattern Recognition*, pages 22647–22657, 2024.
- [70] Xin Zheng and Jianke Zhu. Traj-LO: In defense of LiDAR-only odometry using an effective continuous-time trajectory. *Robotics and Automation Letters*, 9(2):1961–1968, 2024.
- [71] Qian-Yi Zhou, Jaesik Park, and Vladlen Koltun. Fast global registration. In *European conference on computer vision*, pages 766–782. Springer, 2016.
- [72] Minghan Zhu, Maani Ghaffari, and Huei Peng. Correspondence-free point cloud registration with so(3)-equivariant implicit shape representations. In *Proceedings of the 5th Conference on Robot Learning*, pages 1412–1422. PMLR, 2022.
- [73] Minghan Zhu, Maani Ghaffari, William A Clark, and Huei Peng. E2PN: Efficient SE(3)-equivariant point network. In *Conf. on Computer Vision and Pattern Recognition*, pages 1223–1232, 2023.

A. Proof of Lemma 1

Let $\gamma(t) = \mathbf{T} \exp(t\xi^\wedge)$. We write the objective function f in (12) as a function of this perturbation in t as

$$f(\gamma(t)) = \sum_{ij} c_{ij} k(\mathbf{x}_i, \exp(-t\xi^\wedge) \mathbf{T}^{-1} \mathbf{z}_j),$$

where the sum is over $i \in 1, \dots, |X|$ and $j = 1, \dots, |Z|$, respectively. Denote $\mathbf{r}_{ij} := \mathbf{x}_i - \mathbf{T}^{-1} \mathbf{z}_j$ and $f_{ij} := c_{ij} k(\mathbf{x}_i, \mathbf{T}^{-1} \mathbf{z}_j)$ as in (18). Taking the directional derivative, we obtain

$$\begin{aligned} Df(\gamma(t))[\xi^\wedge] &= \sum_{ij} c_{ij} Dk(\mathbf{x}_i, \exp(-t\xi^\wedge) \mathbf{T}^{-1} \mathbf{z}_j)[\xi^\wedge] \\ &= -\frac{1}{2} \sum_{ij} c_{ij} k(\mathbf{x}_i, \mathbf{T}^{-1} \mathbf{z}_j) \lim_{t \rightarrow 0} \frac{1}{t} \{ \\ &\quad \langle \mathbf{x}_i - (\mathbf{I} - t\xi^\wedge + o(t^2)) \mathbf{T}^{-1} \mathbf{z}_j, \\ &\quad \Sigma_{ij}^{-1}(\mathbf{x}_i - (\mathbf{I} - t\xi^\wedge + o(t^2)) \mathbf{T}^{-1} \mathbf{z}_j) \rangle \\ &\quad - \langle \mathbf{x}_i - \mathbf{T}^{-1} \mathbf{z}_j, \Sigma_{ij}^{-1}(\mathbf{x}_i - \mathbf{T}^{-1} \mathbf{z}_j) \rangle \} \\ &= -\sum_{ij} c_{ij} k(\mathbf{x}_i, \mathbf{T}^{-1} \mathbf{z}_j) \langle \mathbf{x}_i - \mathbf{T}^{-1} \mathbf{z}_j, \Sigma_{ij}^{-1} \xi^\wedge (\mathbf{T}^{-1} \mathbf{z}_j) \rangle \\ &= \sum_{ij} f_{ij} \mathbf{r}_{ij}^\top \Sigma_{ij}^{-1} (\mathbf{T}^{-1} \mathbf{z}_j)^\wedge \boldsymbol{\omega} - \sum_{ij} f_{ij} \mathbf{r}_{ij}^\top \Sigma_{ij}^{-1} \mathbf{v}. \end{aligned} \quad (23)$$

Using (3):

$$Df(\mathbf{T})[\xi^\wedge] = \langle \text{grad} f(\mathbf{T}), \xi^\wedge \rangle = g_\omega^\top \boldsymbol{\omega} + g_v^\top \mathbf{v}. \quad (24)$$

and identifying the gradients for translation and rotational components from (23) yields the stated result.

B. Line search

For the Taylor expansion in the line search, we obtain expressions similar to [16, Sec. 7] except that we use anisotropic kernels, with coefficients

$$g_{ij}^1 = \beta_{ij}, \quad (25a)$$

$$g_{ij}^2 = \gamma_{ij} + \frac{1}{2} \beta_{ij}^2, \quad (25b)$$

$$g_{ij}^3 = \delta_{ij} + \beta_{ij} \gamma_{ij} + \frac{1}{6} \beta_{ij}^3, \quad (25c)$$

$$g_{ij}^4 = \varepsilon_{ij} + \beta_{ij} \delta_{ij} + \frac{1}{2} \beta_{ij}^2 \gamma_{ij} + \frac{1}{2} \gamma_{ij}^2 + \frac{1}{24} \beta_{ij}^4. \quad (25d)$$

However, our expressions differ in the following parameters

$$\beta_{ij} = -\langle \boldsymbol{\omega}^\wedge \mathbf{T}^{-1} \mathbf{z}_j + \mathbf{v}, \Sigma_{ij}^{-1}(\mathbf{x}_i - \mathbf{T}^{-1} \mathbf{z}_j) \rangle, \quad (25e)$$

$$\gamma_{ij} = \left(-\frac{1}{2}\right) \langle \boldsymbol{\omega}^\wedge \mathbf{T}^{-1} \mathbf{z}_j + \mathbf{v}, \Sigma_{ij}^{-1}(\boldsymbol{\omega}^\wedge \mathbf{T}^{-1} \mathbf{z}_j + \mathbf{v}) \rangle - \langle (\boldsymbol{\omega}^\wedge)^2 \mathbf{T}^{-1} \mathbf{z}_j + \boldsymbol{\omega}^\wedge \mathbf{v}, \Sigma_{ij}^{-1}(\mathbf{x}_i - \mathbf{T}^{-1} \mathbf{z}_j) \rangle, \quad (25f)$$

$$\delta_{ij} = \left(\langle -\boldsymbol{\omega}^\wedge \mathbf{T}^{-1} \mathbf{z}_j - \mathbf{v}, \Sigma_{ij}^{-1}(\boldsymbol{\omega}^\wedge)^2 \mathbf{T}^{-1} \mathbf{z}_j + \boldsymbol{\omega}^\wedge \mathbf{v} \rangle + \langle -(\boldsymbol{\omega}^\wedge)^3 \mathbf{T}^{-1} \mathbf{z}_j - (\boldsymbol{\omega}^\wedge)^2 \mathbf{v}, \Sigma_{ij}^{-1}(\mathbf{x}_i - \mathbf{T}^{-1} \mathbf{z}_j) \rangle \right), \quad (25g)$$

$$\begin{aligned} \varepsilon_{ij} &= -\frac{1}{2} \left(\langle (\boldsymbol{\omega}^\wedge)^2 \mathbf{T}^{-1} \mathbf{z}_j + \boldsymbol{\omega}^\wedge \mathbf{v}, \Sigma_{ij}^{-1}((\boldsymbol{\omega}^\wedge)^2 \mathbf{T}^{-1} \mathbf{z}_j + \boldsymbol{\omega}^\wedge \mathbf{v}) \rangle + \right. \\ &\quad \left. 2 \langle \boldsymbol{\omega}^\wedge \mathbf{T}^{-1} \mathbf{z}_j + \mathbf{v}, \Sigma_{ij}^{-1}((\boldsymbol{\omega}^\wedge)^3 \mathbf{T}^{-1} \mathbf{z}_j + (\boldsymbol{\omega}^\wedge)^2 \mathbf{v}) \rangle + \right. \\ &\quad \left. 2 \langle (\boldsymbol{\omega}^\wedge)^4 \mathbf{T}^{-1} \mathbf{z}_j + (\boldsymbol{\omega}^\wedge)^3 \mathbf{v}, \Sigma_{ij}^{-1}(\mathbf{x}_i - \mathbf{T}^{-1} \mathbf{z}_j) \rangle \right). \end{aligned} \quad (25h)$$

C. Loss Slices

First, we give intuition on the behavior of the RKHS loss under different perturbations of the ground truth pose. To this end, we sample 8 point clouds from our self-collected LiDAR datasets, later described in Sec. 4.2. Fig. 9 shows the value of the objective function when translating the target point cloud with a pose $\mathbf{T} = \exp(\xi^\wedge)$, where $\xi = (0, \phi, 0, x, 0, z)^\top$ and x, y, ϕ are varied independently over $[-3, 3]$ for translations and $[-0.5, 0.5]$ for rotations. Near the ground truth pose of each pair of point clouds, there is a single maximum of the objective function (12). Besides, the curvature of the loss is greater on the z axis with the kernel choice in (13), resulting in tighter fits along surfaces.

D. Derivation of the explicit form of correction term $\Gamma(\cdot, \cdot)$

Let

$$\xi = \begin{bmatrix} \boldsymbol{\omega} \\ \mathbf{v} \end{bmatrix}, \quad \mathbf{g} = \text{grad} f(\mathbf{T})^\vee = \begin{bmatrix} g_\omega \\ g_v \end{bmatrix}. \quad (26)$$

From (6), the vee representation of the connection term is

$$\Gamma^\vee(\xi^\wedge, \text{grad} f(\mathbf{T})) = \frac{1}{2} (\text{ad}_\xi \mathbf{g} - \text{ad}_\xi^* \mathbf{g} - \text{ad}_\mathbf{g}^* \xi). \quad (27)$$

Using (6),

$$\text{ad}_\xi = \begin{bmatrix} \boldsymbol{\omega}^\wedge & 0 \\ \mathbf{v}^\wedge & \boldsymbol{\omega}^\wedge \end{bmatrix}, \quad \text{ad}_\xi^* = \begin{bmatrix} -\boldsymbol{\omega}^\wedge & -\mathbf{v}^\wedge \\ 0 & -\boldsymbol{\omega}^\wedge \end{bmatrix}. \quad (28)$$

Therefore,

$$\text{ad}_\xi \mathbf{g} = \begin{bmatrix} \boldsymbol{\omega}^\wedge g_\omega \\ \mathbf{v}^\wedge g_\omega + \boldsymbol{\omega}^\wedge g_v \end{bmatrix}, \quad (29)$$

$$\text{ad}_\mathbf{g}^* \xi = \begin{bmatrix} -\boldsymbol{\omega}^\wedge g_\omega - \mathbf{v}^\wedge g_v \\ -\boldsymbol{\omega}^\wedge g_v \end{bmatrix}, \quad \text{ad}_\mathbf{g}^* \xi = \begin{bmatrix} -g_\omega^\wedge \boldsymbol{\omega} - g_v^\wedge \mathbf{v} \\ -g_\omega^\wedge \mathbf{v} \end{bmatrix}. \quad (30)$$

Table 5. *Impact of uniform vs non-uniform downsampling*

KITTI seq. id	G-CVO-2 w. voxel grid		G-CVO-2 w. random	
	trans.	rot.	trans.	rot.
03	1.5743	0.0087	4.0074	0.0172
05	1.0557	0.0060	1.6303	0.0084
06	1.1641	0.0070	1.7082	0.0065
07	1.1106	0.0065	1.0130	0.0082

Tab. 5 shows that G-CVO achieves higher accuracy with voxel-grid downsampling than with random downsampling, likely because the estimated covariance may be noisy with uneven point densities. Nonetheless, G-CVO remains competitive even when point clouds are non-uniform.

Substituting into (27), we obtain

$$\Gamma^\vee(\xi^\wedge, \text{grad}f(\mathbf{T})) = \frac{1}{2} \begin{bmatrix} \omega^\wedge g_\omega + \omega^\wedge g_\omega + v^\wedge g_v + g_\omega^\wedge \omega + g_v^\wedge v \\ v^\wedge g_\omega + 2\omega^\wedge g_v + g_\omega^\wedge v \end{bmatrix}. \quad (31)$$

Using $a^\wedge b = -b^\wedge a$, we have

$$v^\wedge g_v + g_v^\wedge v = 0, \quad \omega^\wedge g_\omega + g_\omega^\wedge \omega = 0, \quad v^\wedge g_\omega + g_\omega^\wedge v = 0. \quad (32)$$

Hence,

$$\Gamma^\vee(\xi^\wedge, \text{grad}f(\mathbf{T})) = \begin{bmatrix} -\frac{1}{2}g_\omega^\wedge \omega \\ -g_v^\wedge \omega \end{bmatrix} = \begin{bmatrix} -\frac{1}{2}g_\omega^\wedge & 0 \\ -g_v^\wedge & 0 \end{bmatrix} \begin{bmatrix} \omega \\ v \end{bmatrix}. \quad (33)$$

Therefore,

$$\Gamma(\xi^\wedge, \text{grad}f(\mathbf{T})) = \left(\begin{bmatrix} -\frac{1}{2}g_\omega^\wedge & 0 \\ -g_v^\wedge & 0 \end{bmatrix} \xi \right)^\wedge. \quad (34)$$

E. Examples of the self-collected datasets

Fig. 10 shows the scenes and the example point cloud observations in Sec. 4.2. The first row is an example frame from the `Dirt Track` sequence, where there is dust floating behind the vehicle, illustrated in red in the LiDAR visualization. The second row is an example frame from the `Race Track` sequence, where there are fewer features to track, and the car travels at greater speeds. The third row shows the `Skid Pad` dataset, a large paved surface.

F. Qualitative tracking results

We provide more qualitative results for frame-to-frame tracking on the KITTI LiDAR dataset and ETH3D RGB-D dataset in Fig. 11 and 12, respectively. We stacked the point clouds based on the poses estimated from the frame-to-frame tracking results in KITTI 00 from Sec. 4.1 and the first 400 frames of ETH3D `table_3` from Sec. 4.3.

G. Ablation on point cloud density

As LiDAR inputs are generally not uniformly dense, we evaluate two types of down-sampling. We consider a voxel grid to achieve a more uniform density, and also evaluate random downsampling, which maintains non-uniformity.

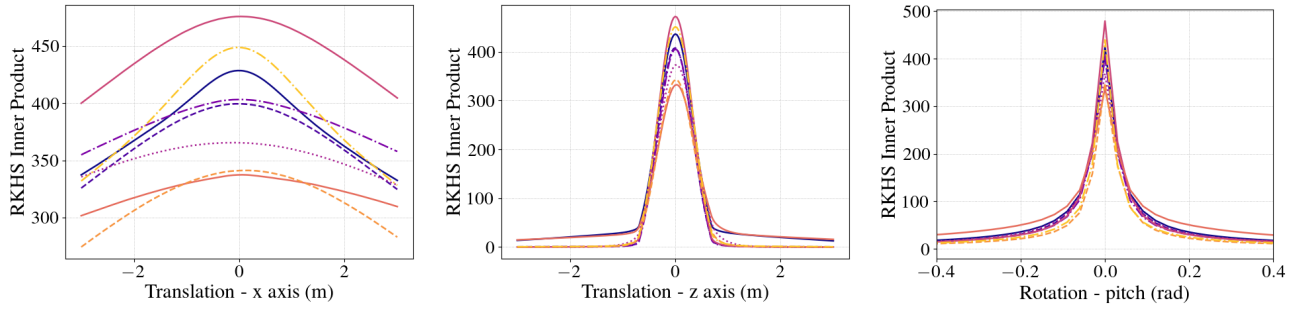


Figure 9. The RKHS inner product with translational and rotational perturbations. Each line corresponds to a different input pair.

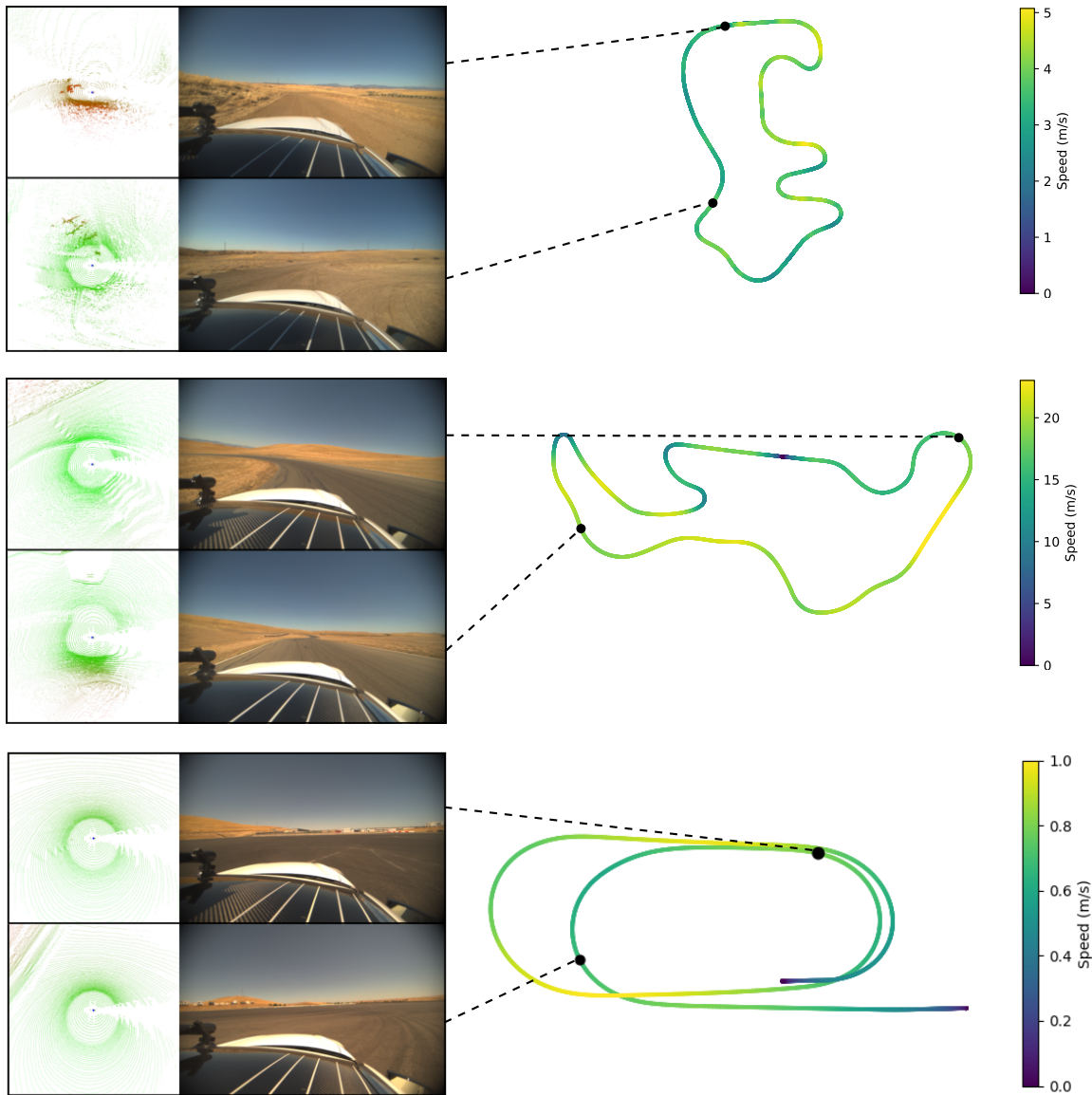


Figure 10. Illustration of the Dirt Track dataset (top), Race Track dataset (middle), and Skid Pad dataset (bottom), with lidar point clouds and images at two points shown at two points along the track.

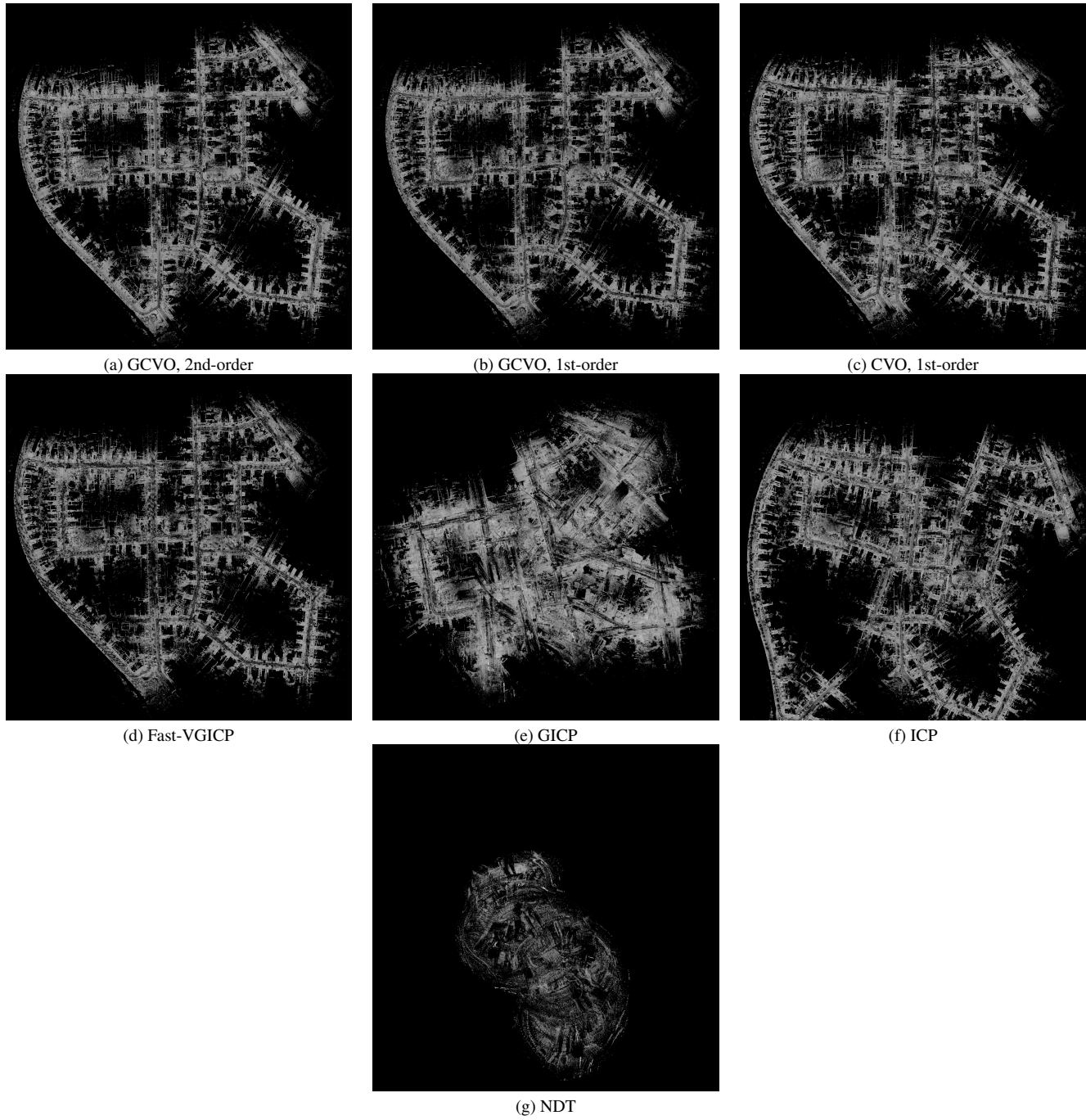
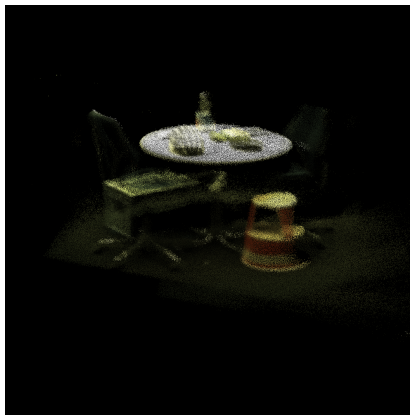
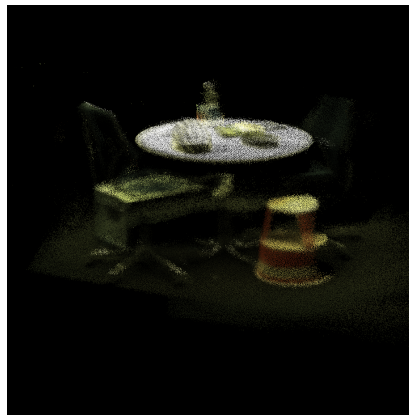


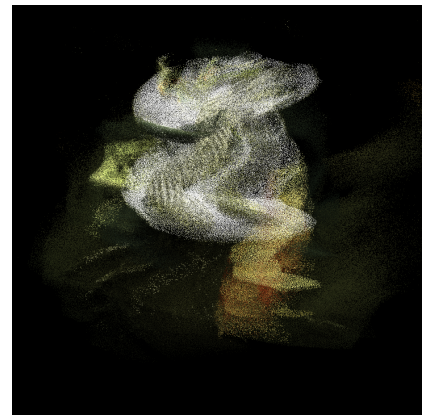
Figure 11. The stacked point clouds using the frame-to-frame tracking poses from KITTI LiDAR sequence 00.



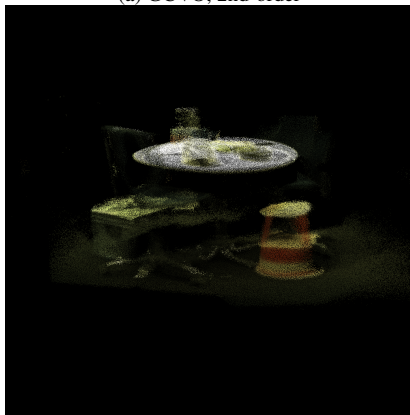
(a) GCVO, 2nd-order



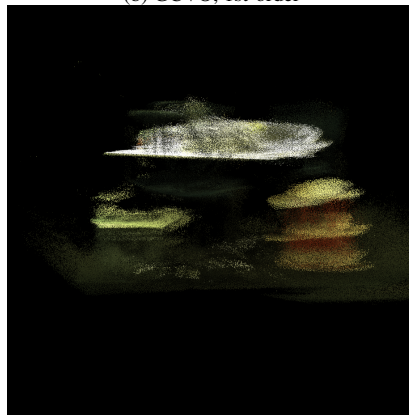
(b) GCVO, 1st-order



(c) Fast-VGICP



(d) GICP



(e) ICP

Figure 12. The stacked point clouds using the frame-to-frame tracking poses from ETH3D RGB-D sequence `table_3`.